Lab Computational Analytics Report

Implementing and Evalutating KADABRA against BRAVA-GNN in Julia

Martin Schlaier

17th July 2026

Summer Semester

First Examiner:
Dr. Aurora Rossi

Second Examiner:

Department of Computer Science
University of Bonn

Declaration of generative AI and AI-assited technologies in the writing process

I hereby declare that generative artificial intelligence tools in the form of, `Gemini 3.5 Flash`, `Gemini 3.5 Pro` were used in the writing process. These tools were solely used to improve language and readability of this work. I edited and reviewed all the generated and modified content as needed and take full responsibility for the content in this work.

Contents

1 Introduction

1.1 Motivation

Identifying highly influential structures within large networks is a fundamental problem in graph analytics. The **Betweenness Centrality Problem (BCP)** formalizes this challenge by calculating the betweenness centrality for the nodes of a graph. This metric quantifies the importance of a node based on the number of shortest paths that pass through it.

Solving the BCP is critical for a wide range of real-world applications. These include identifying critical vulnerabilities in power grids, optimizing data traffic in telecommunication networks, mitigating urban traffic congestion, and securing global supply chains against systemic bottlenecks.

Exact algorithms for the BCP traditionally rely on modern shortest-path frameworks, most notably Brandes' algorithm (Brandes 2001), which executes multiple graph traversals (such as Dijkstra's algorithm). Although these exact methods guarantee optimal solutions, their computational runtime becomes prohibitive on large-scale graphs. Consequently, approximation methods such as **KADABRA** have been developed, successfully trading strict optimality for significantly improved efficiency (Borassi et al. 2019).

Recent advances in Graph Neural Networks (GNNs) show great promise. Specifically, the creators of BRAVA-GNN ('Degree-Mass Message Passing for Betweenness Ranking in Directed and Undirected Networks' 2026) have achieved state-of-the-art performance among GNN approaches, even outperforming KADABRA in full-graph evaluations. However, existing benchmarks primarily compare total graph calculations. A primary advantage of KADABRA is its ability to approximate betweenness centrality exclusively for the top-k nodes, offering substantial speedups by bypassing the rest of the network. This report extends the comparative analysis between KADABRA and BRAVA-GNN by focusing specifically on this top-k approximation performance. Furthermore, this report serves as supplementary documentation for the implementation and integration of the KADABRA algorithm into the Julia `Graphs.jl` ecosystem, as well as comparing the authors C++ implementation vs this new Julia.

1.2 Related Work

The development of polynomial-time exact algorithms for DSP has been a significant area of research. **picard_network_1982** published the first exact algorithm, which was fundamentally built on maximum-flow computations. This was later advanced by **goldberg_finding_1984**, who proposed a version with improved time complexity. A

different approach was taken by **charikar_greedy_2000**, who designed an exact algorithm based on a linear program (LP). The existence of an LP-based algorithm is not surprising, as the max-flow problem can be formulated as an LP. However, Charikar's method offers a new perspective, allowing his greedy-peeling approximation algorithm to be understood as a primal-dual algorithm for DSP. This greedy algorithm, which iteratively removes nodes with the lowest degree, provides a 2-approximation.

Building upon this concept, **iterative peeling algorithms** have been developed. Instead of performing the peeling process once using node degree as the criterion, it is executed multiple times. In each new iteration, more complex node weights, influenced by the results of the previous iterations, are used instead of the simple degree. **boob_flowless_2020** introduced the first algorithm of this type, called *Greedy++*, and demonstrated that it achieves a near-optimal result in just a few iterations and is an order of magnitude faster than Goldberg's maximum flow algorithm. **chekuri_densest_2022** showed that Greedy++ is an approximation scheme that can achieve a value arbitrarily close to the optimum given enough iterations, although the required number of iterations can be extremely high in theory.

hochbaum_flow_2024 introduced the *Incremental Parametric Cut* procedure (IPC), which uses the pseudoflow algorithm**hochbaum_pseudoflow_2008**. This approach uses previous results for initialization when the density parameter changes monotonically, thereby avoiding wasteful recomputations. Techniques like **restartable maximum flow** allow an existing flow solution to be efficiently updated after minor changes to the graph, instead of restarting the algorithm from scratch. **hochbaum_flow_2024** claim that their algorithm is faster by several orders of magnitude than *Greedy++*. However, the publication cited to support this claim is not available online.

Recent advances have produced new parametric flow algorithms with unique capabilities. The Parametric Breadth-First Search (PBFS) (**beines_simpler_2024**) and Parametric Pseudo Flow (PPF) (**Sauer_PPF_2025**) are the first algorithms designed to find a problem's breakpoints—critical parameter values where the minimum cut's structure changes—in strictly monotone order. This specific property opens up new strategies for solving ratio problems like the DSP.

The optimal density for the DSP corresponds to the breakpoint of the highest value. The existence of algorithms that find breakpoints in order enables a new solution strategy: if the problem can be reformulated such that this highest-value breakpoint becomes the *first* to be found, the solution process could be significantly accelerated.

1.3 Our Contribution

A central contribution of this work is the first systematic, empirical comparative study of modern parametric maximum-flow algorithms for the densest subgraph problem (DSP) versus established approximation methods. The investigation covers 47 real-world graphs and includes the first comprehensive and publicly documented evaluation of the Incremental Parametric Cut procedure (IPC) by **hochbaum_flow_2024**, given that their own empirical study is not publicly available. To enable this evaluation we implement

and assess two novel exact algorithms:

DSP-FBA, which leverages recent monotone parametric flow algorithms to find the solution directly by identifying the first breakpoint in a reformulated network, making it the only one required.

DSP-Intersect, an intersection-based algorithm using restartable maximum flow with both push-relabel and pseudoflow implementations, a similar approach to IPC.

Our experiments reveal that the choice of the best algorithm strongly depends on graph properties, particularly degree variance. For graphs with low variance, **DSP-FBA** performs best, whereas for graphs with high variance, IPC is superior. Confirming the hypothesis previously stated by Hochbaum (2024), our results challenge the common assumption that approximation algorithms are always preferable for large graphs. We demonstrate that exact algorithms can achieve optimal solutions with runtimes only slightly higher than those of approximations like Greedy++, making them a viable alternative when solution quality is critical.

Additionally, we systematically analyze the effectiveness of graph pruning based on the greedy algorithm by Charikar (2000). We show that pruning offers substantial benefits, especially for unweighted graphs with high degree variance, and support this with more extensive experiments. Specifically, we investigate the combination of pruning followed by these fast flow algorithms—a configuration not previously analyzed. We show that with pruning, exact algorithms on unweighted graphs can consistently outperform the unpruned Greedy++ approximation, while the benefit is far less pronounced for weighted graphs.

Our implementation extends the parametric maximum flow framework by **beines_simpler_2024** with new, DSP-specific algorithms. These contributions provide both theoretical insights into the performance of exact versus approximate algorithms and practical guidance for practitioners.

1.4 Betweenness Ranking Approximation

The betweenness centrality of a node measures its importance as a bridge or bottleneck based on the fraction of all shortest paths passing through it (**freeman1977set**; Brandes 2001). Given an unweighted graph $G = (V, E)$, let $\sigma_{s,t}$ denote the total number of shortest paths from node s to node t, and let $\sigma_{s,t}(v)$ be the number of those paths that pass through $v \in V$. The exact betweenness centrality $C_B(v)$ of a vertex v is formally defined as:

$$C_B(v) = \sum_{s \neq v \neq t \in V} \frac{\sigma_{s,t}(v)}{\sigma_{s,t}} \tag{1.1}$$

where $\frac{\sigma_{s,t}(v)}{\sigma_{s,t}} = 0$ if $\sigma_{s,t} = 0$ (**brandes2001faster**). In literature and practical network libraries, normalized variants are commonly used to ensure scale-invariance across different graph sizes.

The standard normalization factor depends on whether the source and target node pairs can include v in their normalization pool. For instance, the Julia `Graphs.jl` library normalizes by dividing by the total number of pairs excluding v:

$$C_B(v) = \frac{1}{(n-1)(n-2)} \sum_{s \neq v \neq t \in V} \frac{\sigma_{s,t}(v)}{\sigma_{s,t}} \tag{1.2}$$

Alternatively, the adaptive approximation algorithm KADABRA treats betweenness centrality probabilistically (Borassi et al. 2019). It frames the metric as the exact probability that a shortest path π between two uniformly selected random nodes s and t passes through v (**borassi2019kadabra**). Following the convention where the sum includes cases where $\sigma_{s,t}(v) = 0$ whenever $s = v$ or $t = v$, the normalized centrality is expressed as:

$$C_B(v) = \frac{1}{n(n-1)} \sum_{s \neq t \in V} \frac{\sigma_{s,t}(v)}{\sigma_{s,t}} \tag{1.3}$$

1.4.1 Betweenness Ranking Approximation Problem

Given the prohibitive $O(|V||E|)$ exact computation cost of Brandes' algorithm on massive networks, the goal of the *Betweenness Ranking Approximation Problem* is to learn a size-invariant function $f : V \to \mathbb{R}$ that maps each node to a real-valued score $f(v)$ such that the induced relative ordering maximizes the ordinal correlation with the true centrality hierarchy:

$$\max_{f} \tau \left(\mathcal{R}(f(V)), \mathcal{R}(C_B(V)) \right) \tag{1.4}$$

Where $\mathcal{R}(\cdot)$ represents the rank operator, and τ denotes a ranking index such as the Kendall Tau correlation coefficient.

This formulation allows the estimation to be generalized natively across edge-weighted or directed graph structures. For directed representations, the flow maps independent path dynamics from incoming and outgoing topologies before scoring fusion ($w_{u,v} = 1$ for structural baselines).

For later structural representation, we define the m-th order multi-hop degree mass of a node vector as:

$$d^{(m)} = \left(\sum_{k=0}^{m} A^k \right) d \tag{1.5}$$

as the cumulative structural neighborhood volume within m hops, where A represents the graph adjacency matrix and d is the standard degree vector. In strictly local configurations, $m = 1$ holds.

The problem is equivalent to preserving topological dominance hierarchies without minimizing absolute regression errors. For high-diameter configurations like spatial road networks, structural regularities produce strict discrete symmetry boundaries, causing standard scale-free embeddings to experience over-smoothing. By shifting the inputs to

size-invariant degree masses, long-range structural bottlenecks are captured efficiently within a lightweight parameter footprint.

$$\mathcal{L}(s_u, s_v, y) = \max(0, 1 - y \cdot (s_u - s_v)) \tag{1.6}$$

Since exact deep architectures for this structural proxy require representative generative training, formulations leverage synthetic hyperplanes such as the hyperbolic random graph model to calibrate power-law degree variations outside standard scale-free assumptions prior to message passing.

2 KADABRA

2.1 Overview and Problem Formalization

Exact betweenness centrality (BC) computation is $O(mn)$, which is impractical for large networks. KADABRA shifts to a sampling-based approximation. It samples τ random shortest paths and computes an empirical indicator $X_i(v)$ for each path:

$$X_i(v) = \begin{cases} 1 & \text{if } v \in \pi_i \text{ and } v \neq s, t \\ 0 & \text{otherwise} \end{cases} \tag{2.1}$$

The empirical estimate $\tilde{b}(v)$ is the sample mean of these indicators, providing an unbiased estimator for $bc(v)$.

KADABRA minimizes the sample size τ while ensuring that the estimates for all nodes fall within an absolute error bound λ with a confidence $1 - \delta$:

$$\Pr(\forall v \in V, |\tilde{b}(v) - bc(v)| \leq \lambda) \geq 1 - \delta \tag{2.2}$$

Instead of a static sample size, KADABRA uses **adaptive sampling**, treating τ as a stopping time determined on the fly.

2.2 Algorithmic Innovations

KADABRA optimizes both the time per sample and the total number of samples required.

2.2.1 Balanced Bidirectional BFS (bb-BFS)

To accelerate shortest path sampling, KADABRA uses a balanced bidirectional BFS. It simultaneously grows a forward tree from the source s and a backward tree from the target t, always expanding from the tree with the smaller boundary volume:

$$\text{Expand from } s \iff \sum_{v \in \Gamma^{l_s}(s)} \deg(v) \leq \sum_{u \in \Gamma^{l_t}(t)} \deg(u) \tag{2.3}$$

This cuts path generation time from $\Theta(|E|)$ to $|E|^{\frac{1}{2}+o(1)}$ on realistic random networks.

2.2.2 Rigorous Adaptive Stopping Condition

KADABRA dynamically evaluates lower and upper error bounds (f and g) on the fly using deterministic target confidences. The algorithm terminates as soon as $f \leq \lambda$ and $g \leq \lambda$ for all relevant vertices.

2.3 Theoretical Guarantees

2.3.1 Rigorous Martingale-Based Correctness

Evaluating a stopping condition based on observed data introduces stochastic dependence. KADABRA resolves this by modeling the sequence of path indicators as a martingale. Applying McDiarmid's concentration inequality, it proves that the dynamic stopping condition safely maintains the absolute (λ, δ)-approximation guarantee upon termination.

2.3.2 Adaptive Top-k Centrality Selection

Computing a uniform λ-approximation for all nodes is computationally wasteful. KADABRA instead uses an adaptive relaxation for Top-k ranking. Given a threshold k, KADABRA guarantees that every vertex either has its exact rank established, is formally excluded from the Top-k, or its estimate is within λ error.

It implements this by sorting nodes by empirical centrality and safely terminating only if bounding intervals do not overlap:

- **Internal Separation ($i \leq k$):** Ensures adjacent rank bounds do not overlap.
- **External Exclusion ($i > k$):** Ensures $\tilde{b}(v_k) - f(v_k) \geq \tilde{b}(v_i) + g(v_i)$.

This dynamic boundary validation allows KADABRA to skip unnecessary sampling for low-centrality nodes, enabling fast approximation on massive networks.

3 BRAVA-GNN

3.1 Overview

BRAVA-GNN (Betweenness Ranking Approximation via Degree Mass GNN) is a lightweight graph neural network architecture designed to approximate betweenness centrality rankings for both directed and undirected networks. Its primary contribution is achieving high ranking accuracy while maintaining a parameter count that is strictly independent of the input graph size.

3.2 Key Design Principles

3.2.1 Size-Invariant Node Features

Instead of relying on node-specific embeddings that scale with the number of nodes, BRAVA-GNN uses **multi-hop degree mass** as its input feature. The degree mass captures the cumulative connectivity around a node up to a certain distance. Using degree masses up to order five produces a fixed-dimensional input vector, significantly reducing the model size.

3.2.2 Dual Message Passing

To effectively model directed networks where shortest paths are asymmetric, BRAVA-GNN introduces a dual message-passing mechanism. It applies two parallel streams:

- An **incoming stream** over the adjacency matrix A.
- An **outgoing stream** over the transposed adjacency matrix A^T.

These two streams share weights to keep the architecture compact. The intermediate representations from both directions are then fused multiplicatively, effectively rewarding nodes that are reachable from many sources and can reach many destinations.

3.2.3 Mixed Synthetic Training Distribution

To ensure the model generalizes to diverse real-world topologies without needing domain-specific retraining, BRAVA-GNN is trained entirely on a hybrid dataset of synthetic graphs. The training set includes:

- Directed and undirected scale-free graphs.

- Uniformly directed hyperbolic random graphs (UDHY), which reproduce realistic structural properties like heterogeneous degree distributions and high clustering coefficients.

3.3 Performance

By relying on multi-hop structural features and a compact dual-stream architecture, BRAVA-GNN uses approximately 1,300 parameters—orders of magnitude fewer than previous GNN-based approaches. Despite its small size, it achieves state-of-the-art Kendall τ correlation on real-world test networks, and offers competitive inference latency, achieving massive speedups over exact algorithms.

4 Experimental Setup

4.1 Hardware

The experiments were conducted on a server equipped with an **AMD Ryzen Threadripper 3960X 24-Core Processor** clocked at **3.80 GHz**. The processor features **24 cores**, each with **2 threads**, and is supported by **125,GiB of RAM**. Cache specifications include **768,KiB L1d**, **768,KiB L1i**, **12,MiB L2**, and **128,MiB L3**. The system is additionally equipped with two **NVIDIA GeForce RTX 2080 Ti GPUs**, each with **11,GiB of VRAM**. The operating system used was **Ubuntu 22.04 LTS**.

The original project by Borassi et al. (2019) was implemented in **C++** and we compiled using **CMake version 4.2.1** with **GNU 11.4.0**. Our implementation of KADABRA and BRAVA-GNN was implemented and run with **Julia 1.12.5**. **Simexpal 1.1** (**simexpal2019**) was used to conduct the experiments.

4.2 Instances

For comparability, we use the same instances as 'Degree-Mass Message Passing for Betweenness Ranking in Directed and Undirected Networks' (2026). A variety of instances are obtained from the SNAP database (**leskovec_snap_2014**), ASU's Social Computing Data Repository (**zafarani_social_2009**), and the Koblenz Network Collection (**kunegis_konect_2013**).

All instances used in our experiments are listed in Table **??**

4.3 Experimental Design

To provide a comprehensive evaluation, we divide our experiments into three main categories: accuracy and runtime comparisons, multi-threading scalability, and error bound analysis for approximation algorithms.

4.3.1 Top-k Accuracy and Runtime Comparison

The primary objective of betweenness centrality algorithms in practice is often to identify the top-k most central nodes in a network. We compare our Julia implementation of KADABRA and the trained BRAVA-GNN model against the exact Brandes algorithm, which serves as our ground truth. We also include the original C++ implementation of KADABRA and fast heuristic baselines such as Degree Centrality and PageRank.

instance	n	m	m/n
p2p-Gnutella30	36 682	88 328	2.41
email-Enron	36 692	367 662	10.02
musae-github	37 700	289 003	7.67
gemsec-Facebook	50 515	819 306	16.22
p2p-Gnutella31	62 586	147 892	2.36
soc-Epinions1	75 879	508 837	6.71
soc-Slashdot0811	77 360	905 468	11.70
soc-Slashdot0902	82 168	948 464	11.54
twitch-gamers	168 114	6 797 557	40.43
email-EuAll	265 214	420 045	1.58
com-DBLP	317 080	1 049 866	3.31
web-NotreDame	325 729	1 497 134	4.60
web-BerkStan	685 230	7 600 595	11.09
web-Google	875 713	5 105 039	5.83
com-youtube	1 134 890	2 987 624	2.63
soc-Pokec	1 632 803	30 622 564	18.75
wiki-topcats	1 791 489	28 511 807	15.92
amazon	2 146 057	5 743 146	2.68
wiki-Talk	2 394 385	5 021 410	2.10
com-Orkut	3 072 441	117 185 083	38.14
cit-Patents	3 764 117	16 511 741	4.39
com-lj	3 997 962	34 681 189	8.67
dblp	4 000 148	8 649 011	2.16
soc-LiveJournal1	4 847 571	68 993 773	14.23

Table 4.1: Overview of instances: n is the number of vertices, m the number of edges, Sources: [1] KONECT (**kunegis_konect_2013**), [2] Zafrani (**zafarani_social_2009**); unmarked instances are from SNAP (**leskovec_snap_2014**).

For each instance, we measure the end-to-end runtime (wall-clock time) and memory consumption. To assess the quality of the top-k approximations, we compute the following metrics:

- **Precision at k (Overlap):** The fraction of the true top-k nodes that are present in the predicted top-k set.
- **Kendall's τ:** The rank correlation coefficient computed over the true top-k nodes to measure how well the relative ordering is preserved.
- **NDCG at k:** Normalized Discounted Cumulative Gain, which heavily penalizes ranking errors at the very top of the list.

We evaluate these metrics across various k values (e.g., $k = 10, 50, 100$) to observe how approximation quality degrades as the desired number of central nodes increases.

4.3.2 Multi-Threading Scalability

Since our KADABRA implementation is designed to fully utilize modern multi-core architectures, we conduct a strong scaling experiment. We select a subset of representative graphs and execute the Julia KADABRA implementation with an increasing number of threads $t \in \{1, 2, 4, 8, 16, 24, 32, 48\}$.

By measuring the execution time T_t for t threads, we analyze the parallel speedup $S_t = T_1/T_t$. This allows us to quantify the overhead of thread synchronization and the efficiency of our thread-local memory workspace design, particularly as t approaches the physical and logical core limits of our hardware.

4.3.3 Error Bounds and Parameter Tuning

Approximation algorithms like KADABRA rely heavily on user-defined confidence parameters, specifically the absolute error margin ϵ and the failure probability δ. We perform a grid search over a range of parameter combinations:

$$\epsilon \in \{0.1, 0.05, 0.01, 0.005, 0.001\}$$
$$\delta \in \{0.1, 0.05, 0.01\}$$

For each combination, we measure the empirical runtime and the overall Kendall's τ correlation against the exact betweenness centrality scores. This experiment illustrates the trade-off between strict theoretical guarantees and practical computation time, guiding users in selecting optimal parameters for their specific use cases.

4.4 Results

5 Conclusion

A Absolute Runtimes

Bibliography

Borassi, Michele and Emanuele Natale (20th Feb. 2019). 'KADABRA is an ADaptive Algorithm for Betweenness via Random Approximation'. In: *ACM J. Exp. Algorithmics* 24, 1.2:1–1.2:35. ISSN: 1084-6654. DOI: 10.1145/3284359. URL: https://dl.acm.org/doi/10.1145/3284359 (visited on 13/04/2026).

Brandes, Ulrik (June 2001). 'A faster algorithm for betweenness centrality*'. In: *The Journal of Mathematical Sociology* 25.2, pp. 163–177. ISSN: 0022-250X, 1545-5874. DOI: 10.1080/0022250X.2001.9990249. URL: http://www.tandfonline.com/doi/abs/10.1080/0022250X.2001.9990249 (visited on 13/04/2026).

'Degree-Mass Message Passing for Betweenness Ranking in Directed and Undirected Networks' (2026). In.